S3 — Passo a passo para rodar no servidor (Jetson)

Squad VER-3 · Componente S3 — Alertas e Notificações · Broker do projeto: Kafka da Jetson (172.18.4.56:9092, rede do DComp)
05/11/2026 · Código: main no GitHub, commit 9ba38d5

Resumo: o professor configurou o Kafka na Jetson (**172.18.4.56:9092**, acessível só dentro da rede do DComp) e pediu os componentes prontos para testar e usar com o restante da aplicação a partir de 06/11. O S3 não precisa de mudança no código: o endereço é configuração. Foi preparado para a Jetson: imagem em ARM, instalação do `docker compose` se faltar e um caminho sem internet (imagem construída fora e carregada). **O que falta é o acesso à Jetson** (usuário e SSH); com ele, a instalação é um comando, de dentro da rede do DComp.

Situação

#	Etapa	Situação	Detalhe
1	Código pronto e testado	☑ FEITO	Imagem Docker testada contra o Kafka local; 394 testes passando
2	Script de instalação testado	☑ FEITO	Num Ubuntu 24.04 limpo: instalou o Docker, criou os 9 tópicos num Kafka vazio, ligou o S3 e passou na verificação
3	Verificação mínima no broker	☑ FEITO	tools/verificar_broker.py, dentro da imagem; passou contra Kafka de teste
4	Integração contínua (GitHub Actions)	☑ FEITO	Testes, cobertura e imagem a cada push; passando no GitHub
5	Broker da Jetson	☑ FEITO	E-mail do professor: 172.18.4.56:9092, só na rede do DComp, a partir de 06/11. Nenhuma mudança no código
6	Imagem para ARM (Jetson)	☑ FEITO	05/11: imagem arm64 construída e rodada por emulação; passou na verificação inteira. Arquivo para levar: ods-s3-arm64.tar (47 MB)
7	Juntar na main	☐ FALTA	Ajustes para a Jetson (compose, --sem-construir, documentos)
8	Acesso à Jetson	☐ FALTA	Usuário e SSH, se tem Docker e internet (professor / TRA-1)
9	Entrar na Jetson e rodar o script	☐ FALTA	De dentro da rede do DComp: 3 comandos, uns 3 minutos
10	Conferir e rodar a verificação no broker	☐ FALTA	Na Jetson, logo depois do script; tirar a captura

1. Código pronto e testado ☑ FEITO

Na máquina de desenvolvimento

A imagem foi construída e rodada contra o Kafka local, sem tocar no broker do projeto. Resultado:

Verificação	Resultado
Evento do S2 vira alerta, gravado e publicado	ok
Banco no volume sobrevive ao reinício do contêiner	ok, sem reprocessar nada
`docker stop` (SIGTERM)	sai em ~1 s, fechando o consumidor e o banco
Ao religar, processa um evento novo	em ~5 s
Recursos parado	18 MB de memória, 0,04% de CPU; imagem de 200 MB
Usuário dentro do contêiner	s3 (uid 1000), não root
Suíte de testes	394 testes passando, 98% de cobertura, nenhum usa o broker

2. Script de instalação testado ☑ FEITO

infra/servidor/instalar.sh

Teste: um contêiner Ubuntu 24.04 limpo, sem Docker, fez o papel do servidor, e um Kafka separado, sem nenhum tópico, fez o papel do broker do projeto.

O script	No teste
Instala o Docker, se faltar	instalou (Docker 29.9 e Compose 5.7)
Instala o plugin `docker compose`, se faltar (comum na Jetson)	acrescentado depois do teste, para a Jetson
Guarda o broker em `infra/servidor/.env` (fora do git)	ok; depois de down e up, religou sem export
Cria no broker os tópicos do S3 que faltam, antes de ligar o S3	criou os 9; na segunda execução, "já existe" nos 9
Constrói a imagem e liga o `ods-s3`	ok, contêiner Up
Com `--sem-construir`, usa a imagem já carregada	para a Jetson sem internet (etapa 6)

3. Verificação mínima no broker ☑ FEITO

tools/verificar_broker.py (vai dentro da imagem)

Prova o S3 no broker real com 4 comandos e 1 consulta. Faz o papel do A5 (aplicação do VER-3), sem se passar pelo S2: confere os 9 tópicos, cria um alerta de teste por comando ("[TESTE S3]"), espera o ods.s3.alert, dá ACKNOWLEDGE e CLOSE (o alerta de teste não fica aberto nas aplicações) e consulta o histórico. Fora de localhost, só roda com `--broker-oficial`.

No teste	Resultado
Com o S3 ligado (Kafka de teste, fora de localhost, com --broker-oficial)	tudo ok em ~4 s
Endereço fora de localhost sem --broker-oficial	recusa antes de conectar
Com o S3 desligado	falha em 15 s: "não chegou a resposta... O S3 está ligado?"

4. Integração contínua (GitHub Actions) ☑ FEITO

.github/workflows/ci.yml

A cada push e pull request, o GitHub roda a suíte com cobertura e constrói a imagem. Roda nos servidores do GitHub, sem usar o broker. O resultado de cada push aparece na aba **Actions** do repositório.

5. Broker da Jetson ☑ FEITO

E-mail do professor

Ponto	Situação
Endereço	**172.18.4.56:9092**, acessível só dentro da rede do DComp
Desde quando	testes e uso com o restante da aplicação a partir de 06/11
Mudança no S3	nenhuma no código: o endereço vem do argumento do script (ou de S3_KAFKA_BOOTSTRAP)
Broker anterior	nuvem de testes do Guia do Kafka (34.66.235.91:9093)

6. Imagem para ARM (Jetson) ☑ FEITO

A Jetson é ARM (aarch64), não um PC comum

A imagem parte do `python:3.12-slim`, que existe para ARM, e a `confluent-kafka` tem versão para ARM; o script constrói a imagem na própria Jetson. Use o Docker, não o Python da Jetson: o JetPack 5 vem com Python 3.8, e o S3 precisa de 3.10 ou mais.

No teste em arm64 (05/11, emulação no Docker Desktop)	Resultado
Construir a imagem para linux/arm64	ok, em 96 s
Dentro do contêiner	aarch64, Python 3.12, librdkafka 2.16
Rodar o S3 da imagem arm64 contra o Kafka local e passar na verificação	tudo ok (4 comandos, 1 consulta)
Arquivo para a Jetson sem internet	ods-s3-arm64.tar, 47 MB, nome ods-s3:latest, linux/arm64

Sem internet na Jetson, construa a imagem ARM num PC com Docker e leve o arquivo:

```
# no PC (Docker Desktop), na pasta do s3:
docker buildx build --platform linux/arm64 --output "type=docker,name=ods-s3:latest,dest=ods-s3-arm64.tar" .
# copie ods-s3-arm64.tar e a pasta do s3 para a Jetson; lá:
docker load -i ods-s3-arm64.tar
bash infra/servidor/instalar.sh 172.18.4.56:9092 --sem-construir
```

Plano B, no notebook com Windows (na rede do DComp, com o Docker Desktop aberto). O S3 fica no ar só enquanto o notebook estiver ligado e lá. Testado em 05/11 contra um Kafka sem tópicos:

```
powershell -ExecutionPolicy Bypass -File infra\servidor\ligar_windows.ps1 172.18.4.56:9092
docker exec ods-s3 python tools/verificar_broker.py --broker-oficial
```

7. Juntar na main ☐ FALTA

Na sua máquina

```
git add -A
git commit -m "S3 pronto para a Jetson: imagem ARM, compose e instalação sem internet"
git push
```

Depois, juntar na main (pull request). A Jetson baixa o código da main.

8. Acesso à Jetson ☐ FALTA

Com o professor ou a TRA-1

Pergunta	Por quê
Qual usuário e como entrar (SSH, senha ou chave)?	Sem acesso não há como instalar
Cada squad instala o seu componente na Jetson, ou alguém centraliza?	Para não haver dois S3 ligados ao mesmo broker
A Jetson tem Docker? E o docker compose?	O script instala o compose se faltar, mas precisa de sudo
A Jetson alcança a internet (GitHub, Docker Hub, PyPI)?	Se não, usar o caminho sem internet (etapa 6)

9. Entrar na Jetson e rodar o script ☐ FALTA

De dentro da rede do DComp

```
ssh USUARIO@IP_DA_JETSON                      # usuário e IP: etapa 8
git clone https://github.com/ods-26-2/s3.git && cd s3
bash infra/servidor/instalar.sh 172.18.4.56:9092
```

O script mostra a arquitetura (`aarch64` na Jetson), usa sudo para instalar o que faltar, cria os tópicos e termina com **ok: 9 tópicos do S3 conferidos** e as últimas linhas do log. Para fazer tudo à mão: `docs/IMPLANTACAO_SERVIDOR.md`.

10. Conferir e rodar a verificação no broker ☐ FALTA

Na Jetson, na pasta s3

```
docker compose -f infra/servidor/docker-compose.yml ps          # ods-s3 deve estar "Up"
docker exec ods-s3 python tools/verificar_broker.py --broker-oficial
```

O esperado é terminar com **Tudo certo**; tire a captura: é a evidência do S3.2 no broker real. Depois, quando o S2 ou o A5 mandarem algo, o log mostra `entrada ... resultado=alerta_criado` e `saida ... state=CREATED`.

No dia a dia

Para	Comando, na Jetson (na pasta s3)
Ver o log	`docker compose -f infra/servidor/docker-compose.yml logs -f s3`
Parar (o banco fica guardado)	`docker compose -f infra/servidor/docker-compose.yml down`
Ligar de novo (lê o broker do .env)	`docker compose -f infra/servidor/docker-compose.yml up -d`
Atualizar o código	`bash infra/servidor/instalar.sh 172.18.4.56:9092 --atualizar`
Contar os alertas no banco	`docker exec ods-s3 python -c "import sqlite3; print(sqlite3.connect('/dados/s3.sqlite3').execute('select count(*) from alertas').fetchone())"`

Regras importantes

Um S3 só. Com o S3 no servidor, não ligue outro S3 contra o mesmo broker (nem na sua máquina): dois S3 respondem aos mesmos comandos, cada um com o seu banco. O `kit_aplicacoes.py` e o `teste_kafka_local.py` recusam qualquer broker que não seja localhost, de propósito.

O broker é compartilhado por todos os squads. O S3 espera 1 s entre leituras, sem sobrecarregar a Jetson. A TRA-1 liberou deixar o S3 ligado (05/11).

Problemas comuns

O que aparece	Causa	O que fazer
uso: bash infra/servidor/instalar.sh <broker host:porta>	Faltou o endereço do broker	`bash infra/servidor/instalar.sh 172.18.4.56:9092`
defina S3_KAFKA_BOOTSTRAP	Compose rodado sem o .env (antes do script)	Rodar o script uma vez; ele cria o .env
permission denied ... docker.sock	O usuário ainda não está no grupo docker	Saia e entre de novo pelo ssh, ou use sudo na frente do comando
AVISO: o broker não respondeu	Fora da rede do DComp, ou Kafka da Jetson parado	Rodar de dentro da rede do DComp; testar com `nc -vz 172.18.4.56 9092`; falar com o professor ou a TRA-1
S3 ligado, mas nada chega no log	Ninguém publicando ainda	Normal até o S2 e o A5 publicarem
Depois de um reinício, demora até ~45 s para processar	O S3 anterior foi morto sem fechar (queda de energia, kill)	Esperar: o Kafka libera o lugar sozinho. Com docker stop ou down isso não acontece